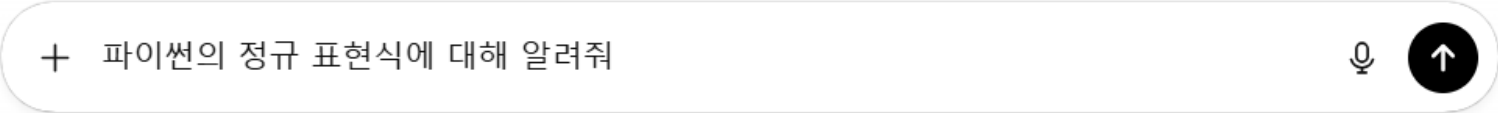
11장. 정규 표현식 및 웹 크롤링

python™

정규식 – Regular Expression

❖ *정규표현식이란?*

- 문자열에서 특정 패턴을 찾거나, 검사하거나, 바꾸는 도구이다



❖ *활용 분야*

- 이메일 형식인지 확인
- 전화번호만 추출
- 숫자만 골라내기
- 특정 단어 치환

정규식 – Regular Expression

- 자주 사용하는 정규 표현식

표현식	설 명
^	정규식 시작
$	정규식 끝
^[0-9]*$	숫자
^[a-zA-Z]*$	영문 대, 소문자
^[가-힣]*$	한글
^010[-](d{3}|\d{4})[-]\d{4}$	휴대폰
^\d{6}[-][1-4]{6}$	주민등록번호

정규식 – Regular Expression

- 정규표현식에 사용되는 메타문자

메타문자	설 명(사용 예)
[ ]	대괄호는 [ ]사이의 문자들과 일치함, [x]
-	문자의 범위를 지정하는 하이픈(-), [1-4]
^	부정을 나타내는 캐럿, [^0-9]
*****	0번 이상 반복, 1번 이상 반복(+)
{m}	m은 반복횟수, {3,4} – 3개 또는 4개
( )	소괄호는 서브 클래스. 그룹을 만들 때 사용
\d	숫자 – [0-9]
\w	알파벳 + 숫자
\s	공백

정규표현식 지원 – re 모듈

- 자주 사용하는 함수

re 모듈 가져오기

import re

메서드	기 능
re.match()	문자열 **처음부터** 패턴이 맞는지
re.serarch()	문자열 **어디든** 패턴이 있으면.
re.findall()	패턴에 맞는 **모든 값 리스트**
re.finditer()	패턴에 맞는 **반복자(iterator)**
re.sub()	패턴을 찾아 **다른 문자열로 치환**

정규식을 사용한 문자열 검색

- 정규 표현식 활용

```
import re

# search 예제
text = "내 전화번호는 010-1234-5678 입니다"
result = re.search(r"\d{3}-\d{4}-\d{4}", text)

print(result.group())

# findall 예제
text = "사과 3개, 배 5개, 감 10개"
numbers = re.findall(r"\d+", text)
print(numbers)

# sub 예제
text = "비밀번호는 1234입니다"
masked = re.sub(r"\d", "*", text)
print(masked)
```

정규표현식 지원 – re 모듈

- 정규 표현식 활용

1. re.compile**('[a-z]+')** : 정규 표현식을 컴파일 한다.
2. match("**korea**") : 문자열의 시작 부분에서 정규 표현식과 일치하는 부분을 찾음

<match 객체의 주요 메서드>

메서드	기 능
group()	매치된 문자열을 돌려준다.
start()	매치된 문자열의 시작위치를 돌려준다.
end()	매치된 문자열의 끝위치를 돌려준다
span()	매치된 문자열의 (시작, 끝)에 해당하는 튜플 반환.

정규표현식 지원 – re 모듈

- re.match()와 re.search() 비교

	match()	search()
검색 위치	문자열의 처음부터 패턴 일치 여부	문자열 전체를 탐색하며 패턴 일치 여부 확인
일치 실패	문자열 중간에 패턴이 있어도 처음이 아니면 None	어디에 있는 처음에 일치하는 것을 반환

정규식을 사용한 문자열 검색

- 정규 표현식 활용

```
import re

# 정규 표현식 패턴 컴파일
# match() 예시
pat = re.compile("[a-z]+")
match = pat.match("Hello World")
if match:
    print("매치된 문자열:", match.group()) #ello

# search() 예시
match = pat.search("Hello World")
if match:
    print("검색된 문자열:", match.group())
```

정규식을 사용한 문자열 검색

- 정규 표현식 활용

```
# '*' 은 0개 이상, '+'는 1개 이상
pat = re.compile("a*b")
match = pat.match("b")
if match:
    print("'*' 매치된 문자열:", match.group())

pat = re.compile("a+b")
match = pat.match("b")
if match:
    print("'+' 매치된 문자열:", match.group())
```

정규식을 사용한 문자열 검색

- 유효성 검사
  - ✓ fullmatch() 함수 – 문자열 전체가 정규 표현식과 일치하는지를 찾음

```
# 전화번호 검증
# phone_pat = re.compile('010-\d{3,4}-\d{4}')
phone_pat = re.compile("010-[0-9]{3,4}-[0-9]{4}")
mat = phone_pat.fullmatch("010-12-5678")
print(bool(mat)) #False

# 한글과 전화번호 패턴 검사
name_pat = "제갈수연";
pat = re.compile("[가-힣]{2,5}")
mat = pat.fullmatch(name_pat)
print(bool(mat)) #True
```

정규식을 사용한 문자열 검색

- 유효성 검사 예제

```
'''def validate_phone_number(phone):
    pattern = r"\d{3}-\d{3,4}-\d{4}"
    if re.fullmatch(pattern, phone):
        return True
    return False'''

def validate_phone_number(phone):
    pattern = r"\d{3}-\d{3,4}-\d{4}"
    return bool(re.fullmatch(pattern, phone))

phone_list = [
    "010-1234-5678",
    "010-123-5678",
    "010-12345-5678",
    "010-1234-56789",
    "01012345678",
]
```

정규식을 사용한 문자열 검색

- 유효성 검사 예제

```
for phone in phone_list:
    if validate_phone_number(phone):
        print(f"{phone}는 유효한 전화번호입니다.")
    else:
        print(f"{phone}는 유효하지 않은 전화번호입니다.")
```

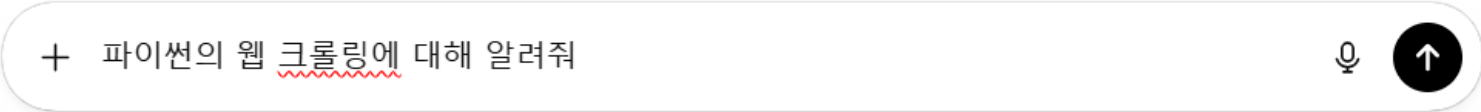
정규식을 사용한 문자열 검색

- 유효성 검사 예제

```
# 한글이름 패턴 유효성 검사
def validate_name(user_name):
    pattern = re.compile("^[가-힣]{2,5}$")
    return bool(pattern.fullmatch(user_name))

while True:
    user_name = input("한글 이름 입력(2~5자): ")

    if validate_name(user_name):
        print(f"이름: {user_name}")
        break
    else:
        print("올바른 한글 이름이 아닙니다. 다시 입력하세요")
```

웹 스크래핑 = 웹 크롤링

➢ ***웹 크롤링이란?***



웹 페이지에서 **자동으로 데이터를 수집**하는 기술입니다.
업무 자동화나 데이터 분석에 정말 많이 쓰여요.

- 뉴스 제목 수집
- 쇼핑몰 상품 가격 모니터링
- 공지사항 자동 수집
- 환율, 날씨 데이터 수집

웹 스크래핑 = 웹 크롤링

- ***웹 크롤링의 기본 구조***

  요청(Request) → 웹 서버 → 응답(Response, HTML) → 파싱(분석) → 데이터 추출

- ***웹 크롤링에 주로 사용되는 라이브러리***

  - requests
  - BeautifulSoup(bs4)

웹 스크래핑 = 웹 크롤링

▷ requests 모듈

Python 프로그래밍 언어용 HTTP 라이브러리이다.

HTTP 프로토콜을 이용하여 웹 사이트로부터 데이터를 송수신할 수 있다.

url 요청 - requests.get(url)

```python
import requests

url = "https://www.python.org"
response = requests.get(url)
print(response) #<Response [200]>
print(response.status_code) #200

html = response.text
print(html)
```

```
<Response [200]>
<Response [404]>
```

정상

페이지 없음

웹 스크래핑 = 웹 크롤링

▷ **requests 모듈(라이브러리)**

```
url2 = "https://www.python.org/3"
response = requests.get(url2)
print(response)
```

```
<Response [200]>
<Response [404]>
```

정상

페이지 없음

로봇 배제 표준

▷ 로봇 배제 표준

로봇 배제 표준이란?

웹사이트에 로봇이 접근하는 것을 방지하기 위한 규약 robots.txt에 기술하고 있다.

- 로봇에 의한 접근이 허용되는 경우라도 웹 서버에 무리가 갈 만큼 반복적으로 웹 페이지를 요청하는 것과 같이 서비스 안정성을 해칠 수 있는 행위를 하지 않아야 함
- 크롤링(또는 스크래핑)으로 취득한 자료를 임의로 배포하거나 변경하는 등의 행위는 저작권을 침해할 수 있으므로 저작권 규정을 준수해야 함

로봇 배제 표준

▷ 로봇 배제 표준

템플릿 태그	설 명
User-agent: * Disallow: /	모든(*) 로봇(검색엔진 봇)에게 루트 디렉터리(/) 이하 모든 문서에 대한 접근을 차단한다.
User-agent: * Allow: /	모든(*) 로봇에게 루트 디렉터리(/) 이하 모든 문서에 대한 접근을 허락한다.
User-agent: * Disallow: /temp/	모든(*) 로봇에게 특정 디렉터리(/temp/)에 대한 접근을 차단한다.

✓ **User-agent: 지침을 적용할 크롤러 이름**

로봇 배제 표준

▷ 로봇 배제 표준 – python.org

https://www.python.org/robots.txt

```
# Directions for robots.  See this URL:
# http://www.robotstxt.org/robotstxt.html
# for a description of the file format.

User-agent: HTTrack
User-agent: puf
User-agent: MSIECrawler
Disallow: /

# The Krugle web crawler (though based on Nutch) is OK.
User-agent: Krugle
Allow: /
Disallow: /~guido/orlijn/
Disallow: /webstats/

# No one should be crawling us with Nutch.
User-agent: Nutch
Disallow: /

# Hide old versions of the documentation and various large
User-agent: *
Disallow: /~guido/orlijn/
Disallow: /webstats/
```

HTTrack, puf, MSIECrawler 같은 **사이트 복제용 툴**이 웹사이트 전체를 긁어가지 못하게 차단하는 역할

로봇 배제 표준

▷ 로봇 배제 표준

```python
import requests

urls = ["https://www.naver.com/", "https://www.python.org/"]
filename = "robots.txt"

for url in urls:
    url_path = url + filename
    print(url_path)
    response = requests.get(url_path)
    print(response)
```

```
https://www.naver.com/robots.txt
<Response [200]>
https://www.python.org/robots.txt
<Response [200]>
```

웹 스크레이핑 = 웹 크롤링

❖ BeautifulSoup 라이브러리(모듈)

HTML과 XML 문서를 파싱하기 위한 파이썬 라이브러리이다.

웹 서버로 부터 HTML 소스코드를 가져온 다음에는 HTML 태그 구조를 해석하기 위한 과정이 필요하다.

HTML 소스 코드를 해석하는 것을 파싱(parsing)이라고 부른다.

▶ BeautifulSoup 설치

pip install BeautifulSoup4

▶ BeautifulSoup 사용

from bs4 import BeautifulSoup

웹 스크레이핑 = 웹 크롤링

❖ BeautifulSoup 라이브러리(모듈)

- ✓ soup.find(태그)

  처음 나오는 태그로 찾기

- ✓ soup.find_all(태그)

  태그에 해당하는 모든 요소 찾아서 리스트로 반환함

- ✓ soup.find(태그, attrs={'class': css_selector})

  태그에 해당하는 선택자로 찾기

선택자(Selector)

➢ 선택자(Selector)란?

스타일의 속성을 적용하는 요소를 '선택자(selector)'라고 부른다. 이 선택자는 태그 하나가 될 수도 있지만 여러 개의 요소를 묶어 별도의 선택자로 지정할 수도 있다.

- **태그 선택자**

  문서에서 특정 태그를 사용한 모든 요소에 스타일 적용

- **id 선택자, class 선택자**

  ✓ **id 선택자** : 문서 안에서 **한 번만 사용**한다면 id 선택자로 정의

  #(샾) 다음에 id 이름 지정

  ✓ **class 선택자** : 문서 안에서 **여러 번 반복할 스타일**이라면 클래스 선택자로 정의.

  마침표(.) 다음에 클래스 이름 지정

선택자(Selector)

➢ 선택자(Selector) 예제

```html
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <title>제품 목록</title>
    <!-- CSS 스타일 추가 -->
    <style>
      h1 { color: blue; }
      p{ font-size: 18px;}
      .p1 { color: red; }
      .p2 { font-family: "궁서"; }
      .p3 { font-weight: bold; }
    </style>
</head>
<body>
    <h1>마늘</h1>
    <h2>마늘의 효능</h2>
    <p class="p1">마늘은 면역력 강화에 도움을 줍니다.</p>
    <p class="p2">마늘은 심혈관 건강에 좋습니다.</p>
    <p class="p3">마늘은 항산화 작용을 합니다.</p>
</body>
</html>
```

서울시청 웹 크롤링하기

✓ 서울 시청 사이트 크롤링하기

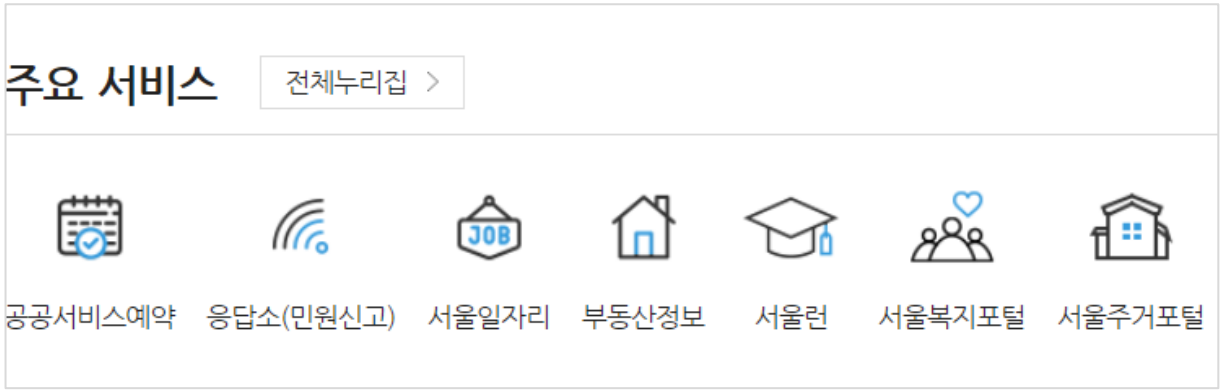
```
from bs4 import BeautifulSoup

url = "https://www.seoul.go.kr/main/index.jsp"
response = requests.get(url)

soup = BeautifulSoup(response.text, 'html.parser')
print(soup.title)
```

✓ 0.5s

```
<title>동행·매력 특별시 서울 | 서울특별시</title>
```

서울시청 웹 크롤링하기

✓ 메뉴 글자 수집하기



웹 브라우저 >우클릭 > 검사(단축키 F12)

```
▼<div class="m_service">  == $0
  ▼<ul> flex
    ▼<li class="public">
      ▼<a href="//yeyak.seoul.go.kr" onclick="action_logging({tr_code:'serv
       1'});" target="_blank" title="새창">
         <i class="ico_service"></i>
         "공공서비스예약"
       </a>
     </li>
    ▼<li class="answer">
      ▶<a href="//eungdapso.seoul.go.kr" onclick="action_logging({tr_code:'
```

서울시청 웹 크롤링하기

✓ 메뉴 글자 수집하기

```
# 메뉴 글자 추출
# find() : 조건에 맞는 첫 번째 요소 반환
first_li = soup.find('li', attrs={'class': 'public'})
print(first_li)
print(first_li.text)
# print(first_li.get_text())
```

✓ 0.0s

```
<li class="public">
<a href="//yeyak.seoul.go.kr" onclick="action_logging({tr_code:'service01'});
</li>

공공서비스예약
```

서울시청 웹 크롤링하기

✓ 메뉴 글자 수집하기

```python
# find_all() : 조건에 맞는 모든 요소 반환
div = soup.find('div', attrs={'class': 'm_service'})
all_li = div.find_all('li')
# print(all_li)

for li in all_li:
    print(li.text)

print(all_li[1].text) #응답소(민원신고)
```

✓ 0.0s

```
공공서비스예약


응답소(민원신고)


서울일자리
```

웹 스크레이핑 = 웹 크롤링

❖ **주요 검색 함수**

- ✓ **select_one(태그이름.선택자이름)**

  첫번째 요소로 찾기

- ✓ **select(태그이름.선택자이름 > 하위 태그)**

  태그에 해당하는 모든 요소 찾기

서울시청 웹 크롤링하기

✓ select_one()

```
# select_one() : CSS 선택자로 조건에 맞는 첫 번째 요소 반환
first_li = soup.select_one('div.m_service > ul > li.public')
print(first_li)
print(first_li.text)
```

✓ 0.0s

```
<li class="public">
<a href="//yeyak.seoul.go.kr" onclick="action_logging({tr_code:'serv
</li>

공공서비스예약
```

서울시청 웹 크롤링하기

✓ select()

```
# select() : CSS 선택자로 요소 반환
li = soup.select('div.m_service > ul > li')
for item in li:
    print(item.text)
```

✓ 0.0s

```
공공서비스예약


응답소(민원신고)


서울일자리
```

실습1. 국립중앙박물관 관람 정보

- 국립 중앙 박물관 관람 정보

홈페이지 : https://www.museum.go.kr/

관람 정보 > 관람 안내 > 검사(F12)

관람시간	**월, 화, 목, 금, 일요일**: 10:00 ~ 18:00 (입장 마감: 17:30) **수, 토요일**: 10:00 ~ 21:00 (입장 마감: 20:30) · 옥외 전시장(정원)은 오전 7시부터 오후 10시까지 관람하실 수 있습니다.

관람료	무료	상설전시관, 어린이박물관, 무료 특별전시 해당
	유료	유료 특별전시 해당 **관람권 구입하는 곳**: 특별전시실 1 앞 매표소 **관람권 판매시간**: 관람 종료 30분 전까지

실습1. 국립중앙박물관 관람 정보

- 국립 중앙 박물관 관람 정보

```python
import requests
from bs4 import BeautifulSoup

# 국립 중앙 박물관
url = "https://www.museum.go.kr/MUSEUM/contents/M0101000000.do?menuId=tour-guidance"
response = requests.get(url)
html = BeautifulSoup(response.text, 'html.parser')

# 관람 안내
first_ul = html.select_one('ul.display-content')
print(first_ul.get_text())
```

```
관람시간

월, 화, 목, 금, 일요일:  9:30 - 17:30 (입장 마감: 17:00)
수, 토요일: 9:30 - 21:00 (입장 마감: 20:30)

옥외 전시장(정원)은 오전 7시부터 오후 10시까지 관람하실 수 있습니다.
어린이박물관
월~일요일 9:30 - 17:20 회차별 예약제 운영
어린이박물관 예약 바로가기
```

실습1. 국립중앙박물관 관람 정보

- 국립 중앙 박물관 관람 정보

```
# 관람 안내
contents = html.select('ul.display-content-area > li > ul')
'''for content in contents:
    print(content.get_text())'''

# 관람료
print(contents[2].get_text())
```

✓ 0.0s

```
관람료


무료

상설전시관, 어린이박물관, 무료 특별전시 해당

유료

유료 특별전시 해당
관람권 구입하는 곳: 특별전시실 1 앞 매표소 (특별전시실 2 는 전시실 입구에서도 구매 가능)
관람권 판매시간: 관람 종료 30분 전까지
```

KBS 뉴스 기사

- 뉴스 기사 크롤링하기

홈페이지 : https://news.kbs.co.kr/

KBS > 뉴스 > 메인 기사

KBS 뉴스 기사

➢ 뉴스 기사 크롤링하기

```python
import requests
from bs4 import BeautifulSoup

url = "https://news.kbs.co.kr/news/pc/view/view.do?ncd=8539528"
response = requests.get(url)

html = BeautifulSoup(response.text, "html.parser")

# 기사 제목 추출
title = html.select_one("h4.headline-title").text.strip()
print("기사 제목:", title)
```

✓ 0.6s

```
기사 제목: 이란, 호르무즈 개방했다 하루 만에 다시 통제…호르무즈 두고 기싸움
```

KBS 뉴스 기사

➢ 뉴스 기사 크롤링하기

```python
# 기사 본문 추출
content = html.select_one("div.detail-body")
print(content.text.strip())
```

✓ 0.0s

```
[앵커]중동 정세의 핵심 변수인 호르무즈 해협이 예측불허의 방향으로 움직이고 있습니다.이란이 호르무즈 개방을 발표한 지 하루도 안 돼 다시 '봉쇄'를 선언했습니다.미국이 상응한 조치를 취하고 있지 않다며 돌연 입장을 바꾼 겁니다.열었다 닫았다, 호르무즈를 둘러싼 치열한 수싸움 먼저 강푸른 기자가 보도합니다. [리포트] 이스라엘과 레바논의 극적인 휴전 합의에, 호르무즈 해협 개방을 선언한 이란 아라그치 외무장관. 레바논 휴전 상황을 반영해 휴전 기간  모든 상선의 항해를 전면 허용한다고 밝혔지만, 이는 채 하루를 넘기지 못했습니다. 이란 군은 성명을 통해 미국이 약속을 어겨, 선의로 개방했던 호르무즈 해협을 "이전 상태로 복구했다"고 주장했습니다.  [이란 국영 방송 : "미국은 전에도 그랬듯 반복적으로 약속을 어기고 소위 '봉쇄'라는 명목으로 약탈과 해적 행위를 계속하고 있습니다."] 트럼프 대통령이  해협 개방 발표 직후, 환영하면서도, "종전 협상이 끝날 때까지 이란에 대한 봉쇄는 전면 유지하겠다"고 밝힌 걸 정면 비판한 겁니다.  [트럼프/미국 대통령 : "(종전) 합의서에 서명하면 봉쇄가 해제됩니다. 합의서에 서명하는 즉시 봉쇄가 해제될 겁니다."] 미국과의 2차 담판을 앞두고, 협상력을 극대화하기 위해 군부가 방침을 뒤집은 것으로 해석됩니다. 실제로 해협에서 녹음된 이란군 무전에서도 관련 경고가 포착되는 등, 재폐쇄 선언 전까지 해협을 빠져나온 선박은  극소수뿐인 것으로 추정됩니다. [이란군 VHF 교신음 : "해협을 통과하고자 한다면, 반드시 이란 혁명수비대 해군의 허가를 받아야 한다."] 결국, 호르무즈 해협 개방 소식에 급등했던 전 세계 주요 증시와  중동 정세 낙관론은  다시금 불안정한 상황에 빠졌습니다. KBS 뉴스 강푸른입니다.  영상편집:이상미/그래픽:고석훈/자료조사:전가영
```

네이버 금융 크롤링하기

- **환율정보 수집하기**

  네이버 > 증권 > 시장지표 > 환전 고시 환율

네이버 금융 크롤링하기

- 환율정보 수집하기

```python
# 네이버 환율 정보
import requests
from bs4 import BeautifulSoup

# 네이버 환율 정보 페이지 URL
url = "https://finance.naver.com/marketindex/"

# 페이지 요청 및 HTML 파싱
response = requests.get(url)
html = BeautifulSoup(response.text, "html.parser")
print(html)
```

네이버 금융 크롤링하기

- **환율정보 수집하기**

```
<div class="market_data">
<div class="market1">
<div class="title">
<h2 class="h_market1"><span>환전 고시 환율</span></h2>
</div>
```

```python
# 환율 정보 제목 추출
title = html.select_one("h2.h_market1").text.strip()
print("환율 정보 제목:", title)
```

✓ 0.0s

```
환율 정보 제목: 환전 고시 환율
```

네이버 금융 크롤링하기

- 환율정보 수집하기

```
<h3 class="h_lst"><span class="blind">미국 USD</span></h3>
<div class="head_info point_dn">
<span class="value">1,467.80</span>
<span class="txt_krw"><span class="blind">원</span></span>
```

```python
# 환율 정보 리스트 추출 - select()
all_li = html.select("div.market1 ul li")
print(all_li)

# 환율 종류 - select_one()
exchange_name = html.select_one("span.blind").text.strip()
print("환율 종류:", exchange_name)

# 환율 가격 - select_one()
exchange_price = html.select_one("span.value").text.strip()
print("환율 가격:", exchange_price)
```

네이버 금융 크롤링하기

- **환율정보 수집하기**

```python
# 환율 종류 및 가격 출력
for li in all_li:
    exchange_name = li.select_one("span.blind").text.strip()
    exchange_price = li.select_one("span.value").text.strip()
    # print(f"{exchange_name}: {exchange_price}")
    print(f"{exchange_name.split()[0]}: {exchange_price}")
```

✓ 0.0s

```
미국: 1,467.80
일본: 925.44
유럽연합: 1,726.72
중국: 215.30
```